Deep Learning School

Физтех-Школа Прикладной математики и информатики (ФПМИ) МФТИ

Some parts of the notebook are almost the copy of [mmta-team course](). Special thanks to mmta-team for making them publicly available. [Original notebook]().

Прочитайте семинар, пожалуйста, для успешного выполнения домашнего задания. В конце ноутка напишите свой вывод. Работа без вывода оценивается ниже.

Задача поиска схожих по смыслу предложений

Мы будем ранжировать вопросы StackOverflow на основе семантического векторного представления

До этого в курсе не было речи про задачу ранжировния, поэтому введем математическую формулировку

Задача ранжирования(Learning to Rank)

- X - множество объектов
- $X^l = \{x_1, x_2, \ldots, x_l\}$ - обучающая выборка

На обучающей выборке задан порядок между некоторыми элементами, то есть нам известно, что некий объект выборки более релевантный для нас, чем другой:

- $i \prec j$ - порядок пары индексов объектов на выборке X^l с индексами i и j

Задача:

построить ранжирующую функцию $a : X \to R$ такую, что

$$i \prec j \Rightarrow a(x_i) < a(x_j)$$

Embeddings

Будем использовать предобученные векторные представления слов на постах Stack Overflow.
[A word2vec model trained on Stack Overflow posts]

```
!wget https://zenodo.org/record/1199620/files/SO_vectors_200.bin?download=1
```

```
--2026-03-01 16:23:53--  https://zenodo.org/record/1199620/files/SO_vectors_2
Resolving zenodo.org (zenodo.org)... 188.184.98.114, 188.184.103.118, 188.185
Connecting to zenodo.org (zenodo.org)|188.184.98.114|:443... connected.
HTTP request sent, awaiting response... 301 MOVED PERMANENTLY
Location: /records/1199620/files/SO_vectors_200.bin [following]
--2026-03-01 16:23:54--  https://zenodo.org/records/1199620/files/SO_vectors_
Reusing existing connection to zenodo.org:443.
HTTP request sent, awaiting response... 200 OK
Length: 1453905423 (1.4G) [application/octet-stream]
Saving to: ‘SO_vectors_200.bin?download=1’

SO_vectors_200.bin? 100%[===================>]   1.35G  12.6MB/s    in 2m 1s

2026-03-01 16:25:55 (11.5 MB/s) - ‘SO_vectors_200.bin?download=1’ saved [1453
```

```
!pip install gensim
from gensim.models.keyedvectors import KeyedVectors
wv_embeddings = KeyedVectors.load_word2vec_format("SO_vectors_200.bin?downlo
```

```
Requirement already satisfied: gensim in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dis
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packag
```

Как пользоваться этими векторами?

Посмотрим на примере одного слова, что из себя представляет embedding

```
word = 'dog'
if word in wv_embeddings:
    print(wv_embeddings[word].dtype, wv_embeddings[word].shape)
```

```
float32 (200,)
```

```
print(f"Num of words: {len(wv_embeddings.index_to_key)}")
```

```
Num of words: 1787145
```

Найдем наиболее близкие слова к слову `dog`:

Вопрос 1:

- Входит ли слово `cat` в топ-5 близких слов к слову `dog`? Какое место оно занимает?

```
# method most_simmilar
'''your code'''
wv_embeddings.most_similar('dog',topn=5)
```

```
[('animal', 0.8564180135726929),
 ('dogs', 0.7880866527557373),
 ('mammal', 0.7623804211616516),
 ('cats', 0.7621253728866577),
 ('animals', 0.760793924331665)]
```

Ответ: 'cat' не входит в топ-5 ближайших слов к 'dog'.

Векторные представления текста

Перейдем от векторных представлений отдельных слов к векторным представлениям вопросов, как к **среднему** векторов всех слов в вопросе. Если для какого-то слова нет предобученного вектора, то его нужно пропустить. Если вопрос не содержит ни одного известного слова, то нужно вернуть нулевой вектор.

```
import numpy as np
import re
# you can use your tokenizer
# for example, from nltk.tokenize import WordPunctTokenizer
class MyTokenizer:
    def __init__(self):
        pass
    def tokenize(self, text):
        return re.findall('\w+', text)
tokenizer = MyTokenizer()
```

```
<>:9: SyntaxWarning: invalid escape sequence '\w'
<>:9: SyntaxWarning: invalid escape sequence '\w'
/tmp/ipython-input-39012/2760772041.py:9: SyntaxWarning: invalid escape seque
  return re.findall('\w+', text)
```

```
def question_to_vec(question, embeddings, tokenizer, dim=200):
    """
        question: строка
        embeddings: наше векторное представление
        dim: размер любого вектора в нашем представлении

        return: векторное представление для вопроса
    """

    '''your code'''
    tokens = tokenizer.tokenize(question.lower())
    vectors = [embeddings[word] for word in tokens if word in embeddings]
    if vectors:
```

```
            return np.mean(vectors, axis=0)
        return np.zeros(dim)
```

Теперь у нас есть метод для создания векторного представления любого предложения.

Вопрос 2:

- Какая третья (с индексом 2) компонента вектора предложения `I love neural networks` (округлите до 2 знаков после запятой)?

```
# Предложение
question = "I love neural networks"

'''your code'''
vector = question_to_vec(question, wv_embeddings, tokenizer)
print(f"Третья компонента вектора: {vector[2]:.2f}")
```

```
Третья компонента вектора: -1.29
```

Чтобы изменить содержимое ячейки, дважды нажмите на нее (или выберите "Ввод")

Оценка близости текстов

Представим, что мы используем идеальные векторные представления слов. Тогда косинусное расстояние между дублирующими предложениями должно быть меньше, чем между случайно взятыми предложениями.

Сгенерируем для каждого из N вопросов R случайных отрицательных примеров и примешаем к ним также настоящие дубликаты. Для каждого вопроса будем ранжировать с помощью нашей модели $R+1$ примеров и смотреть на позицию дубликата. Мы хотим, чтобы дубликат был первым в ранжированном списке.

Hits@K

Первой простой метрикой будет количество корректных попаданий для какого-то K:

$$\text{Hits@K} = \frac{1}{N}\sum_{i=1}^{N}[rank_q_i^{'} \leq K],$$

- $[x < 0] \equiv \begin{cases} 1, & x < 0 \\ 0, & x \geq 0 \end{cases}$ - индикаторная функция
- q_i - i-ый вопрос
- $q_i^{'}$ - его дубликат

- $rank_q_i^{'}$ - позиция дубликата в ранжированном списке ближайших предложений для вопроса q_i.

Hits@K измеряет долю вопросов, для которых правильный ответ попал в топ-K позиций среди отранжированных кандидатов.

DCG@K

Второй метрикой будет упрощенная DCG метрика, учитывающая порядок элементов в списке путем домножения релевантности элемента на вес равный обратному логарифму номера позиции::

$$\text{DCG@K} = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{\log_2(1+rank_q_i^{'})}\cdot[rank_q_i^{'} \leq K],$$

С такой метрикой модель штрафуется за большой ранк корректного ответа.

DCG@K измеряет качество ранжирования, учитывая не только факт наличия правильного ответа в топ-K, но и ***его точную позицию***.

Пример оценок

Вычислим описанные выше метрики для игрушечного примера. Пусть

- $N = 1, R = 3$
- "Что такое python?" - вопрос q_1
- "Что такое язык python?" - его дубликат $q_i^{'}$

Пусть модель выдала следующий ранжированный список кандидатов:

1. "Как изучить c++?"
2. "Что такое язык python?"
3. "Хочу учить Java"
4. "Не понимаю Tensorflow"

$\Rightarrow rank_q_i^{'} = 2$

Вычислим метрику *Hits@K* для *K = 1, 4*:

- [K = 1] $\text{Hits@1} = [rank_q_i^{'} \leq 1]$

Проверяем условие $\text{rank}_{q_1'} \leq 1$: ***условие неверно***.

Следовательно, $[\text{rank}_{q_1'} \leq 1] = 0$.

- [K = 4] $\text{Hits@4} = [rank_q_i^{'} \leq 4] = 1$

Проверяем условие $\text{rank}_{q_1'} \leq 4$: ***условие верно***.

Вычислим метрику *DCG@K* для *K = 1, 4*:

- [K = 1] $\text{DCG@1} = \frac{1}{\log_2(1+2)} \cdot [2 \leq 1] = 0$
- [K = 4] $\text{DCG@4} = \frac{1}{\log_2(1+2)} \cdot [2 \leq 4] = \frac{1}{\log_2 3}$

Вопрос 3:

- Вычислите `DCG@10`, если $rank_q_i^{'} = 9$(округлите до одного знака после запятой)
- `DCG@10` = $1/log2(1+9) = 0.3$

Более сложный пример оценок

Рассмотрим пример с $N > 1$, где $N = 3$ (три вопроса) и для каждого вопроса заданы позиции их дубликатов. Вычислим метрики **Hits@K** для разных значений K.

- $N = 3$: Три вопроса (q_1, q_2, q_3).
- Для каждого вопроса известна позиция его дубликата ($\text{rank}_{q_i'}$):
  - $\text{rank}_{q_1'} = 2$,
  - $\text{rank}_{q_2'} = 5$,
  - $\text{rank}_{q_3'} = 1$.

Мы будем вычислять **Hits@K** для $K = 1, 5$.

Для $K = 1$:

Подставим значения:

$$\text{Hits@1} = \frac{1}{3} \cdot \left([\text{rank}_{q_1'} \leq 1] + [\text{rank}_{q_2'} \leq 1] + [\text{rank}_{q_3'} \leq 1]\right).$$

Проверяем условие $\text{rank}_{q_i'} \leq 1$ для каждого вопроса:

- $\text{rank}_{q_1'} = 2 \to 2 \nleq 1 \to 0$,
- $\text{rank}_{q_2'} = 5 \to 5 \nleq 1 \to 0$,
- $\text{rank}_{q_3'} = 1 \to 1 \leq 1 \to 1$.

Сумма:

$$\text{Hits@1} = \frac{1}{3} \cdot (0 + 0 + 1) = \frac{1}{3}.$$

$$\boxed{\text{Hits@1} = \frac{1}{3}}.$$

Для $K = 5$:

Подставим значения:

$$\text{Hits@5} = \frac{1}{3} \cdot \left([\text{rank}_{q_1'} \leq 5] + [\text{rank}_{q_2'} \leq 5] + [\text{rank}_{q_3'} \leq 5]\right).$$

Проверяем условие $\text{rank}_{q_i'} \leq 5$ для каждого вопроса:

- $\text{rank}_{q_1'} = 2 \to 2 \leq 5 \to 1$,
- $\text{rank}_{q_2'} = 5 \to 5 \leq 5 \to 1$,
- $\text{rank}_{q_3'} = 1 \to 1 \leq 5 \to 1$.

Сумма:

$$\text{Hits@5} = \frac{1}{3} \cdot (1 + 1 + 1) = 1.$$

$$\boxed{\text{Hits@5} = 1}.$$

Теперь вычислим метрику **DCG@K** для того же примера, где $N = 3$ (три вопроса), и для каждого вопроса известна позиция его дубликата ($\text{rank}_{q_i'}$):

- $\text{rank}_{q_1'} = 2$,
- $\text{rank}_{q_2'} = 5$,
- $\text{rank}_{q_3'} = 1$.

Мы будем вычислять **DCG@K** для $K = 1, 5$.

Для $K = 1$: Подставим значения:

$$\text{DCG@1} = \frac{1}{3} \cdot \left(\frac{1}{\log_2(1 + \text{rank}_{q_1'})} \cdot [\text{rank}_{q_1'} \leq 1] + \frac{1}{\log_2(1 + \text{rank}_{q_2'})} \cdot [\text{rank}_{q_2'}$$

Проверяем условие $\text{rank}_{q_i'} \leq 1$ для каждого вопроса:

- $\text{rank}_{q_1'} = 2 \to 2 \nleq 1 \to 0$,
- $\text{rank}_{q_2'} = 5 \to 5 \nleq 1 \to 0$,
- $\text{rank}_{q_3'} = 1 \to 1 \leq 1 \to 1$.

Сумма:

$$\text{DCG@1} = \frac{1}{3} \cdot (0 + 0 + 1) = \frac{1}{3}.$$

$$\boxed{\text{DCG@1} = \frac{1}{3}}.$$

Для $K = 5$: Подставим значения:

$$\text{DCG@5} = \frac{1}{3} \cdot \left(\frac{1}{\log_2(1 + \text{rank}_{q_1'})} \cdot [\text{rank}_{q_1'} \leq 5] + \frac{1}{\log_2(1 + \text{rank}_{q_2'})} \cdot [\text{rank}_{q_2'}$$

Проверяем условие $\text{rank}_{q_i'} \leq 5$ для каждого вопроса:

- $\text{rank}_{q_1'} = 2 \to 2 \leq 5 \to 1$,
- $\text{rank}_{q_2'} = 5 \to 5 \leq 5 \to 1$,
- $\text{rank}_{q_3'} = 1 \to 1 \leq 5 \to 1$.

Сумма:

$$\text{DCG@5} = \frac{1}{3} \cdot (0.631 + 0.387 + 1) = \frac{1}{3} \cdot 2.018 \approx 0.673.$$

$$\boxed{\text{DCG@5} \approx 0.673}.$$

Вопрос 4:

- Найдите максимум `Hits@47 - DCG@1`?
- Если ранг будет от 2 до 47, то разность будет равна `1`, это и будет максимальное значение

HITS_COUNT и DCG_SCORE

Каждая функция имеет два аргумента: dup_ranks и k.

dup_ranks является списком, который содержит рейтинги дубликатов (их позиции в ранжированном списке).

К примеру для "Что такое язык python?" $dup_ranks = [2]$.

```
def hits_count(dup_ranks, k):
    """
        dup_ranks: list индексов дубликатов
        k: пороговое значение для ранга
        result: вернуть Hits@k
    """
    # Подсчитываем количество дубликатов, чей ранг <= k
    '''your code'''
    hits_value = sum([1 if rank <= k else 0 for rank in dup_ranks])/len(dup_
    return hits_value
```

```
dup_ranks = [2]

k = 1
hits_value = hits_count(dup_ranks, k)
print(f"Hits@1 = {hits_value}")
```

```
k = 4
hits_value = hits_count(dup_ranks, k)
print(f"Hits@4 = {hits_value}")
```

```
Hits@1 = 0.0
Hits@4 = 1.0
```

```
import math

def dcg_score(dup_ranks, k):
    """
        dup_ranks: list индексов дубликатов
        k: пороговое значение для ранга
        result: вернуть DCG@k
    """
    # Вычисляем сумму для всех релевантных дубликатов
    '''your code'''
    dcg_score = sum([1/math.log2(1+rank) if rank <= k else 0 for rank in dup
    # Делим на общее количество вопросов
    '''your code'''
    dcg_value = dcg_score/len(dup_ranks)
    return dcg_value
```

```
# Пример списка позиций дубликатов
dup_ranks = [2]

# Вычисляем DCG@1
dcg_value = dcg_score(dup_ranks, k=1)
print(f"DCG@1 = {dcg_value:.3f}")

# Вычисляем DCG@4
dcg_value = dcg_score(dup_ranks, k=4)
print(f"DCG@10 = {dcg_value:.3f}")
```

```
DCG@1 = 0.000
DCG@10 = 0.631
```

Протестируем функции. Пусть $N = 1$, то есть один эксперимент. Будем искать копию вопроса и оценивать метрики.

```
import pandas as pd
```

```
copy_answers = ["How does the catch keyword determine the type of exception

# наши кандидаты
candidates_ranking = [["How Can I Make These Links Rotate in PHP",
                       "How does the catch keyword determine the type of exc
                       "NSLog array description not memory address",
                       "PECL_HTTP not recognised php ubuntu"],]

# dup_ranks — позиции наших копий, так как эксперимент один, то этот массив
dup_ranks = [2]
```

```
# вычисляем метрику для разных k
print('Ваш ответ HIT:', [hits_count(dup_ranks, k) for k in range(1, 5)])
print('Ваш ответ DCG:', [round(dcg_score(dup_ranks, k), 5) for k in range(1,
```

```
Ваш ответ HIT: [0.0, 1.0, 1.0, 1.0]
Ваш ответ DCG: [0.0, 0.63093, 0.63093, 0.63093]
```

У вас должно получиться

```
# correct_answers - метрика для разных k
correct_answers = pd.DataFrame([[0, 1, 1, 1], [0, 1 / (np.log2(3)), 1 / (np.
                               index=['HITS', 'DCG'], columns=range(1,5))
correct_answers
```

	1	2	3	4
HITS	0	1.00000	1.00000	1.00000
DCG	0	0.63093	0.63093	0.63093

Данные

arxiv link

`train.tsv` - выборка для обучения.
В каждой строке через табуляцию записаны: **<вопрос>, <похожий вопрос>**

`validation.tsv` - тестовая выборка.
В каждой строке через табуляцию записаны: **<вопрос>, <похожий вопрос>, <отрицательный пример 1>, <отрицательный пример 2>, ...**

```
!unzip stackoverflow_similar_questions.zip
```

```
Archive:  stackoverflow_similar_questions.zip
replace data/.DS_Store? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
  inflating: data/.DS_Store
replace __MACOSX/data/._.DS_Store? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
  inflating: __MACOSX/data/._.DS_Store
  inflating: data/train.tsv
  inflating: data/validation.tsv
```

Считайте данные.

```
def read_corpus(filename):
    data = []
    with open(filename, encoding='utf-8') as file:
        for line in file:
            data.append(line.strip().split('\t'))
    return data
```

Нам понадобиться только файл validation.

```
validation_data = read_corpus('./data/validation.tsv')
```

Кол-во строк

```
len(validation_data)
```

```
3760
```

Размер нескольких первых строк

```
for i in range(25):
    print(i + 1, len(validation_data[i]))
```

```
1 1001
2 1001
3 1001
4 1001
5 1001
6 1001
7 1001
8 1001
9 1001
10 1001
11 1001
12 1001
13 1001
14 1001
15 1001
16 1001
17 1001
18 1001
19 1001
20 1001
21 1001
22 1001
23 1001
24 1001
25 1001
```

Ранжирование без обучения

Реализуйте функцию ранжирования кандидатов на основе косинусного расстояния. Функция должна по списку кандидатов вернуть отсортированный список пар (позиция в исходном списке кандидатов, кандидат). При этом позиция кандидата в полученном списке является его рейтингом (первый - лучший). Например, если исходный список кандидатов был [a, b, c], и самый похожий на исходный вопрос среди них - c, затем a, и в конце b, то функция должна вернуть список **[(2, c), (0, a), (1, b)]**.

```
from sklearn.metrics.pairwise import cosine_similarity
from copy import deepcopy
```

```
def rank_candidates(question, candidates, embeddings, tokenizer, dim=200):
    """
        question: строка
        candidates: массив строк(кандидатов) [a, b, c]
        result: пары (начальная позиция, кандидат) [(2, c), (0, a), (1, b)]
    """
    question_vector = question_to_vec(question, embeddings, tokenizer, dim).
    candidate_vectors = np.array([question_to_vec(candidate, embeddings, tok
    similarity_scores = cosine_similarity(question_vector, candidate_vectors
    result = sorted(enumerate(candidates), key=lambda x: similarity_scores[x

    return result
```

Протестируйте работу функции на примерах ниже. Пусть $N = 2$, то есть два эксперимента

```
questions = ['converting string to list', 'Sending array via Ajax fails']

candidates = [['Convert Google results object (pure js) to Python object', #
               'C# create cookie from string and send it',
               'How to use jQuery AJAX for an outside domain?'],

              ['Getting all list items of an unordered list in PHP',      #
               'WPF- How to update the changes in list item of a list',
               'select2 not displaying search results']]
```

```
for question, q_candidates in zip(questions, candidates):
        ranks = rank_candidates(question, q_candidates, wv_embeddings, token
        print(ranks)
        print()
```

```
[(1, 'C# create cookie from string and send it'), (0, 'Convert Google results

[(0, 'Getting all list items of an unordered list in PHP'), (2, 'select2 not
```

Для первого эксперимента вы можете полностью сравнить ваши ответы и правильные ответы. Но для второго эксперимента два ответа на кандидаты будут **скрыты**(*)

```
# должно вывести
results = [[(1, 'C# create cookie from string and send it'),
            (0, 'Convert Google results object (pure js) to Python object'),
            (2, 'How to use jQuery AJAX for an outside domain?')],
           [(0, 'Getting all list items of an unordered list in PHP'), #скры
            (2, 'select2 not displaying search results'), #скрыт
            (1, 'WPF- How to update the changes in list item of a list')]] #
```

Последовательность начальных индексов вы должны получить `для эксперимента 1` 1, 0, 2.

Вопрос 5:

- Какую последовательность начальных индексов вы получили `для эксперимента 2`(перечисление без запятой и пробелов, например, `102` для первого эксперимента?
- `021` - верная последовательность для 2 эксперимента

Теперь мы можем оценить качество нашего метода. Запустите следующие два блока кода для получения результата. Обратите внимание, что вычисление расстояния между векторами занимает некоторое время (примерно 10 минут). Можете взять для validation 1000 примеров.

```
from tqdm.notebook import tqdm
```

```
wv_ranking = []
max_validation_examples = 1000
for i, line in enumerate(tqdm(validation_data)):
    if i == max_validation_examples:
        break
    q, *ex = line
    ranks = rank_candidates(q, ex, wv_embeddings, tokenizer)
    wv_ranking.append([r[0] for r in ranks].index(0) + 1)
```

27% 1000/3760 [01:08<02:46, 16.61it/s]

```
for k in tqdm([1, 5, 10, 100, 500, 1000]):
    print("DCG@%4d: %.3f | Hits@%4d: %.3f" % (k, dcg_score(wv_ranking, k), k
```

```
100%                                                  6/6  [00:00<00:00,  280.97it/s]
DCG@   1: 0.415 | Hits@   1: 0.415
DCG@   5: 0.502 | Hits@   5: 0.582
DCG@  10: 0.525 | Hits@  10: 0.651
DCG@ 100: 0.570 | Hits@ 100: 0.874
DCG@ 500: 0.583 | Hits@ 500: 0.973
DCG@1000: 0.586 | Hits@1000: 1.000
```

Из формул выше можно понять, что

- $\mathrm{Hits@K}$ **монотонно неубывающая функция** K, которая стремится к 1 при $K \to \infty$.
- $\mathrm{DCG@K}$ **монотонно неубывающая функция** K, но рост замедляется с увеличением K из-за убывания веса $\frac{1}{\log_2(1+\mathrm{rank}_{q_i'})}$.

Эмбеддинги, обученные на корпусе похожих вопросов

```
train_data = read_corpus('./data/train.tsv')
```

Улучшите качество модели.
Склеим вопросы в пары и обучим на них модель Word2Vec из gensim. Выберите размер window. Объясните свой выбор.

Рассмотрим подробнее данное склеивание.

1. Каждая строка из train_data разбивается на вопрос (question) и список кандидатов.
2. Для каждого кандидата вопрос склеивается с ним в одну строку.
3. Склеенная строка (combined_text) токенизируется, и полученный список токенов добавляется в общий корпус (corpus).

Пример

```
Вопрос: "What is Python?"
Кандидаты: ["Python is a programming language", "Java is another language"]
Склеенные строки:
    "What is Python? Python is a programming language"
    "What is Python? Java is another language"

Токенизированные списки:
    ['what', 'is', 'python', 'python', 'is', 'a', 'programming', 'language']
    ['what', 'is', 'python', 'java', 'is', 'another', 'language']
```

```
train_data[111258]
```

```
['Determine if the device is a smartphone or tablet?',
 'Change imageView params in all cards together']
```

```
# Создаем общий корпус текстов
corpus = []

'''your code'''
for line in train_data:
    q = line[0]
    candidates = line[1:]
    for cand in candidates:
        combined_text = f"{q} {cand}"
        corpus.append(tokenizer.tokenize(combined_text.lower()))
```

```
from gensim.models import Word2Vec
embeddings_trained = Word2Vec(
    sentences=corpus,        # Корпус токенизированных текстов
    vector_size=200,         # Размерность векторов
    window=5,                # Размер окна контекста
    min_count=2,             # Минимальная частота слов
    workers=4                # Количество потоков
).wv
```

```
wv_ranking = []
max_validation_examples = 1000
for i, line in enumerate(tqdm(validation_data)):
    if i == max_validation_examples:
        break
    q, *ex = line
    ranks = rank_candidates(q, ex, embeddings_trained, tokenizer)
    wv_ranking.append([r[0] for r in ranks].index(0) + 1)
```

```
 27%                                                  1000/3760 [01:11<03:11, 14.39it/s]
```

```
for k in tqdm([1, 5, 10, 100, 500, 1000]):
    print("DCG@%4d: %.3f | Hits@%4d: %.3f" % (k, dcg_score(wv_ranking, k), k
```

```
100%                                                 6/6 [00:00<00:00, 402.13it/s]
DCG@   1: 0.317 | Hits@   1: 0.317
DCG@   5: 0.390 | Hits@   5: 0.455
DCG@  10: 0.415 | Hits@  10: 0.533
DCG@ 100: 0.468 | Hits@ 100: 0.788
DCG@ 500: 0.488 | Hits@ 500: 0.943
DCG@1000: 0.494 | Hits@1000: 1.000
```

Замечание:

Решить эту задачу с помощью обучения полноценной нейронной сети будет вам предложено, как часть задания в одной из домашних работ по теме "Диалоговые системы".

Напишите свой вывод о полученных результатах.

- Какой принцип токенизации даёт качество лучше и почему?
- Помогает ли нормализация слов?
- Какие эмбеддинги лучше справляются с задачей и почему?
- Почему получилось плохое качество решения задачи?
- Предложите свой подход к решению задачи.

Вывод:

- Токенизация с использованием регулярных выражений (как в классе MyTokenizer), которая очищает текст от пунктуации и оставляет только буквенно-цифровые символы, дает более стабильное качество. Это уменьшает размер словаря и убирает «шум» (знаки препинания), который не несет семантической нагрузки для модели Word2Vec, позволяя эмбеддингам фокусироваться на значимых токенах.
- Приведение к нижнему регистру и лемматизация (или стемминг) существенно улучшают результат, тк размер словаря уменьшается, и модель фокусируются на значимых токенах
- "Теоретически, эмбеддинги, обученные на целевом корпусе (парах дубликатов), должны справляться лучше. Однако в нашем эксперименте предобученная модель Word2Vec показала более высокие метрики (DCG@10 0.525 против 0.415). Это связано с колоссальной разницей в объемах обучающих данных: универсальная модель обучалась на полном дампе Stack Overflow (словарь 1.7 млн слов), в то время как наша кастомная модель — лишь на 100 тысячах пар. На таком малом объеме данных модель не успела сформировать достаточно богатое семантическое пространство."
- Основная причина получение эмбеддинга предложения через среднее эмбеддингов слов, тк смысл предложения не может быть равен сумме смыслов его слов, некоторые слова могут значить больше чем другие.
- Чтобы решить проблему, которая описана выше можно использовать трансформер BERT, в котором есть несколько encoder слоев, и служебный токен CLS, который будет давать смысл предложения путём не простого среднего.